

Setup & Upload

```
# =====
# Multiple Linear Regression - Multi-Channel Marketing Analysis
# =====

!pip install pandas seaborn statsmodels matplotlib scipy -q

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from statsmodels.formula.api import ols
from statsmodels.stats.outliers_influence import variance_inflation_factor
import warnings
warnings.filterwarnings('ignore')

print("✅ Libraries imported!")

# Upload dataset
from google.colab import files
print("📁 Please upload 'marketing_sales_data.csv'")
uploaded = files.upload()

filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)
print(f"✅ Dataset loaded! Shape: {df.shape}")
df.head()
```

✅ Libraries imported!

📁 Please upload 'marketing_sales_data.csv'

marketing_sales_data.csv

marketing_sales_data.csv(text/csv) - 26651 bytes, last modified: 6/15/2026 - 100% done

Saving marketing_sales_data.csv to marketing_sales_data.csv

✅ Dataset loaded! Shape: (572, 5)

	TV	Radio	Social Media	Influencer	Sales	
0	Low	3.518070	2.293790	Micro	55.261284	
1	Low	7.756876	2.572287	Mega	67.574904	
2	High	20.348988	1.227180	Micro	272.250108	
3	Medium	20.108487	2.728374	Mega	195.102176	
4	High	31.653200	7.776978	Nano	273.960377	

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

Data Exploration & Cleaning

```
print("=== Dataset Info ===")
print(df.info())
print("\n=== Missing Values ===")
print(df.isna().sum())

# Drop missing values
df_clean = df.dropna().copy()
print(f"\nCleaned Shape: {df_clean.shape}")
```

```
=== Dataset Info ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 572 entries, 0 to 571
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   TV               572 non-null    object
1   Radio            572 non-null    float64
2   Social Media     572 non-null    float64
3   Influencer       572 non-null    object
4   Sales            572 non-null    float64
dtypes: float64(3), object(2)
memory usage: 22.5+ KB
None
```

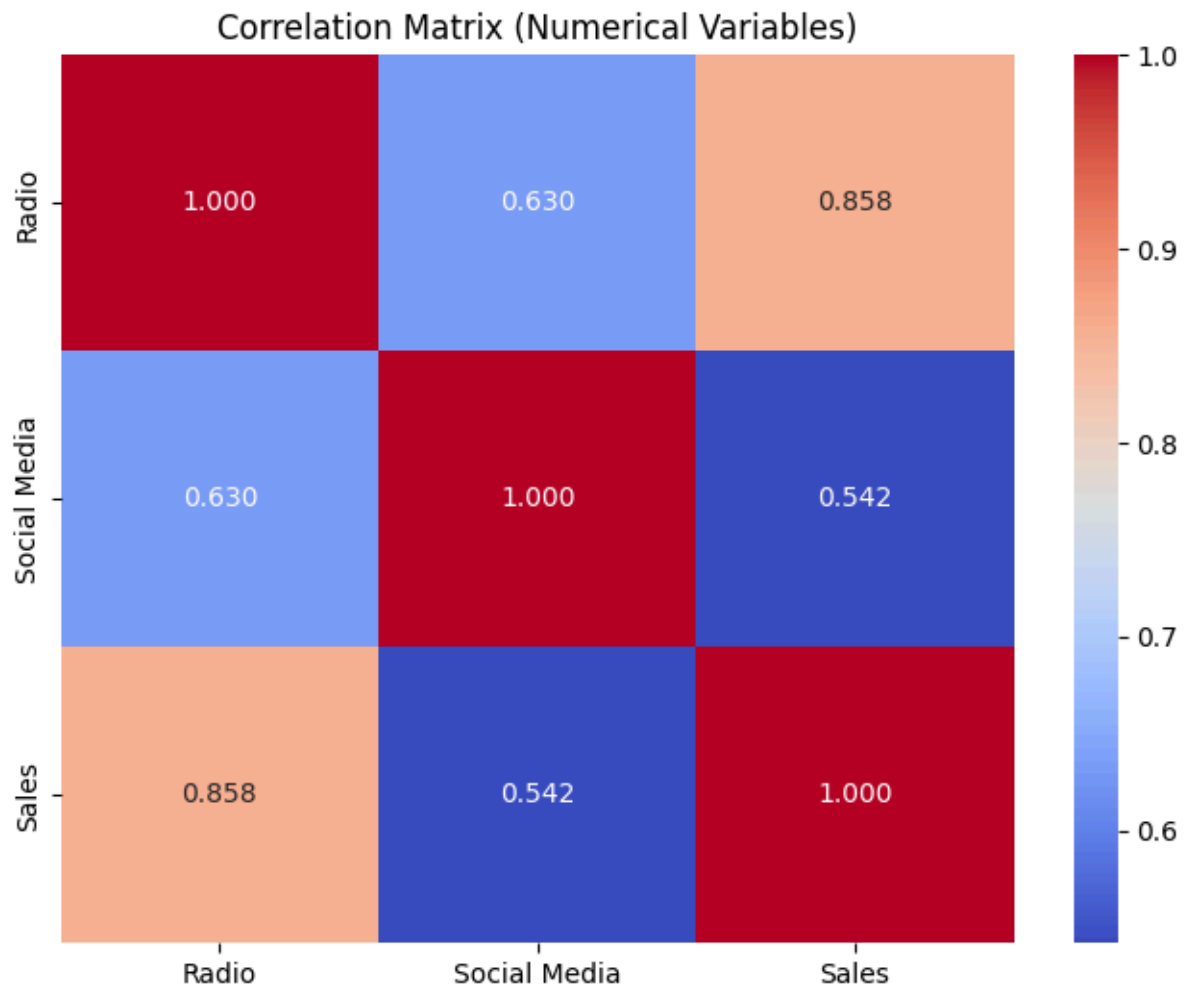
```
=== Missing Values ===
TV          0
Radio       0
Social Media 0
Influencer  0
Sales       0
dtype: int64
```

```
Cleaned Shape: (572, 5)
```

Exploratory Data Analysis + Multicollinearity Check Python

```
# Numerical columns correlation
numerical_cols = ['Radio', 'Social Media', 'Sales']
plt.figure(figsize=(8, 6))
sns.heatmap(df_clean[numerical_cols].corr(), annot=True, cmap='coolwarm', fmt='.3f')
plt.title("Correlation Matrix (Numerical Variables)")
plt.show()

# VIF for multicollinearity
X_num = df_clean[['Radio', 'Social Media']]
X_num = sm.add_constant(X_num)
vif_data = pd.DataFrame()
vif_data["Feature"] = X_num.columns
vif_data["VIF"] = [variance_inflation_factor(X_num.values, i) for i in range(X_num.shape[1])]
print("Variance Inflation Factor (VIF):")
print(vif_data)
```



Variance Inflation Factor (VIF):

	Feature	VIF
0	const	4.713305
1	Radio	1.657894
2	Social Media	1.657894

Build Multiple Linear Regression Model

```
# Using all numerical predictors + handling categorical if needed
# For simplicity and project requirement: Radio + Social Media as main predictors

df_clean = df_clean.rename(columns={'Social Media': 'Social_Media'})
model = ols('Sales ~ Radio + Social_Media', data=df_clean).fit()

print("=== Multiple Linear Regression Model Summary ===")
print(model.summary())
```

```
=== Multiple Linear Regression Model Summary ===
                        OLS Regression Results
=====
Dep. Variable:          Sales    R-squared:                0.736
Model:                  OLS      Adj. R-squared:            0.735
Method:                 Least Squares    F-statistic:          794.1
Date:                  Sun, 21 Jun 2026    Prob (F-statistic):    2.18e-165
```

```

Time: 14:06:01 Log-Likelihood: -3003.1
No. Observations: 572 AIC: 6012.
Df Residuals: 569 BIC: 6025.
Df Model: 2
Covariance Type: nonrobust
=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept    43.8100      4.197     10.438      0.000     35.566     52.054
Radio         8.2843      0.268     30.893      0.000      7.758      8.811
Social_Media  0.1022      1.113      0.092      0.927     -2.084      2.288
=====
Omnibus:            0.891   Durbin-Watson:           1.843
Prob(Omnibus):      0.640   Jarque-Bera (JB):           0.778
Skew:               0.086   Prob(JB):                 0.678
Kurtosis:           3.053   Cond. No.                  43.9
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly spe

Diagnostic Plots (Assumption Checking)

```

fitted = model.fittedvalues
residuals = model.resid

plt.figure(figsize=(14, 10))

plt.subplot(2, 2, 1)
sns.scatterplot(x=fitted, y=residuals)
plt.axhline(0, color='red', linestyle='--')
plt.title('Residuals vs Fitted (Linearity & Homoscedasticity)')

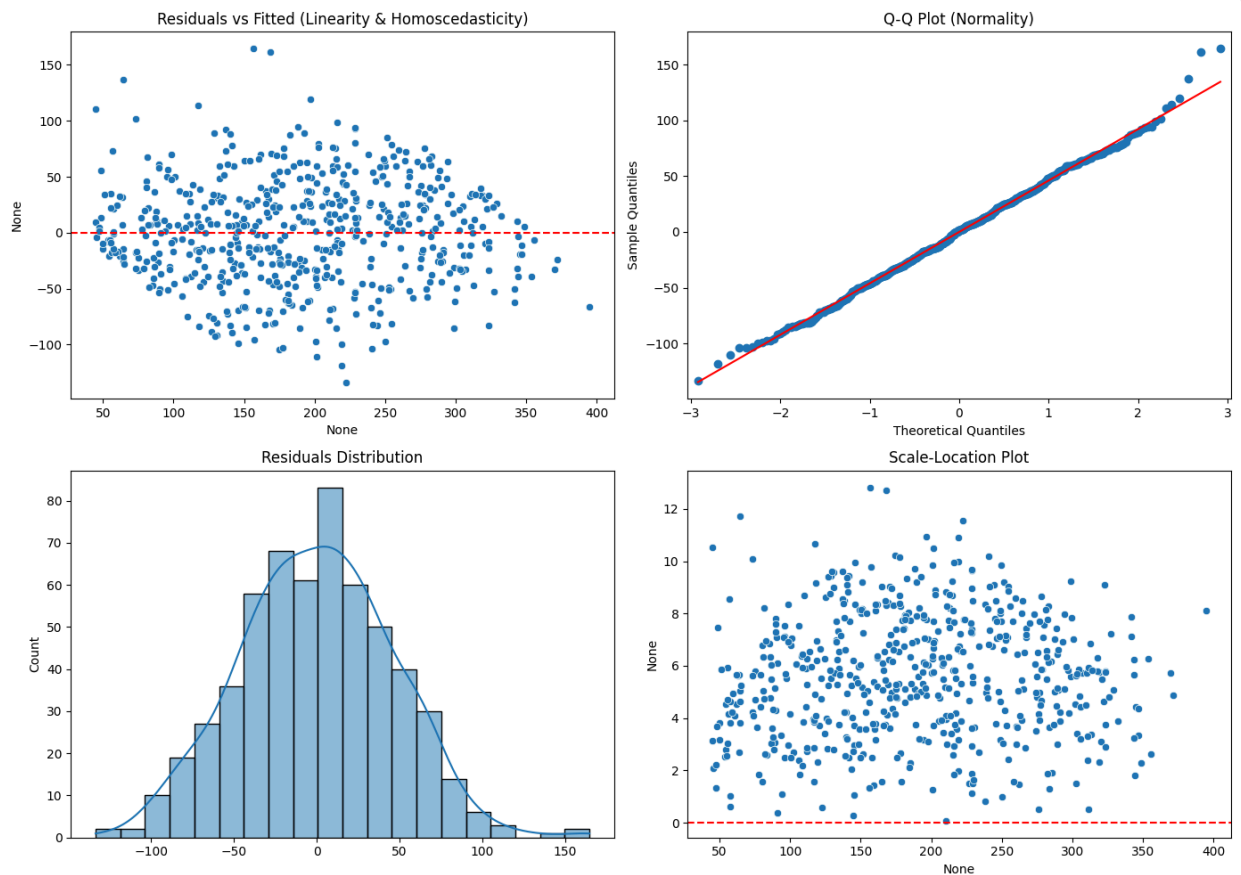
plt.subplot(2, 2, 2)
sm.qqplot(residuals, line='s', ax=plt.gca())
plt.title('Q-Q Plot (Normality)')

plt.subplot(2, 2, 3)
sns.histplot(residuals, kde=True)
plt.title('Residuals Distribution')

plt.subplot(2, 2, 4)
sns.scatterplot(x=fitted, y=np.sqrt(np.abs(residuals)))
plt.axhline(0, color='red', linestyle='--')
plt.title('Scale-Location Plot')

plt.tight_layout()
plt.show()

```



Interpretation & Business Recommendation

```
p_radio:.4f}))")
```

```
p-value: {p_social:.4f}))")
```

```
ted with ${coef_radio:.2f}M increase in Sales.")
```

```
ef_social:.2f}M per $1M).")
```

```
est driver. Allocate more budget to Radio while monitoring Social Media efficiency."
```

```
=== 📊 MODEL INTERPRETATION ===
```

```
Adjusted R-squared: 0.7353
```

```
Coefficient for Radio: 8.2843 (p-value: 0.0000)
```

 BUSINESS RECOMMENDATION:

- Holding other channels constant,
Each additional \$1M in Radio advertising is associated with \$8.28M increase in Sales
- Recommendation: Prioritize **Radio** as the strongest driver. Allocate more budget